SIMD-Accelerated Matrix Multiplication for Neural-Network Workloads

0. Abstract

Matrix multiplication is a fundamental computational operation used in many neural-network architectures, including protein language models, time-series forecasting models, and deep Q-networks. This project proposes the implementation and performance evaluation of sequential and SIMD-parallel matrix multiplication to demonstrate data-level parallelism.

Our project aims to develop a sequential implementation in C/C++ and a vectorized implementation using handwritten ARM64 NEON SIMD assembly, combined with cache blocking (tiling) and register reuse. Since each of our group members uses Apple Silicon MacBooks and does not have guaranteed access to NVIDIA GPUs or compatible x86-64 hardware, our implementation will use ARM NEON assembly as an approved alternative to the AVX-family implementations specified in the assignment. Each NEON register holds four 32-bit floating-point values, so the SIMD version computes four output elements in one fused multiply-add instruction, giving a theoretical maximum speedup of 4x.

Both implementations will multiply the same input matrices. Their results will be compared to validate correctness, and their execution times will be measured across different matrix sizes. The project will calculate speedup and analyze how vectorization, register reuse, cache blocking, problem size, and memory access affect performance. The expected output is a working proof of concept that demonstrates SIMD-based data-level parallelism and provides a quantitative comparison between sequential and vectorized execution.

1. Description of the Proposed Project

1.1 Project Overview

The objective of this project is to demonstrate how SIMD instructions, combined with cache blocking and register reuse, can improve the execution of matrix multiplication by processing multiple numerical values within a single vector instruction.

The project is relevant to our group's planned research directions:

- **Protein language models** – Transformer architectures use matrix operations to transform protein sequence representations.
- **Depression deterioration monitoring and forecasting** – Neural forecasting models can use matrix operations to process representations derived from wearable sensor data.

- **Deep Q-networks** – Neural networks used in reinforcement learning perform matrix operations when transforming state representations into action-value estimates.

Instead of implementing three separate thesis systems, our group will focus on matrix multiplication as a shared computational primitive. This provides a manageable proof of concept that is relevant to all three research directions without requiring model training, specialized datasets, or access to dedicated GPU infrastructure.

The project will compare a conventional sequential implementation against an explicitly vectorized implementation and evaluate their correctness and performance. It will run on a single CPU core so that the measured speedup comes only from SIMD, register reuse, and cache blocking, and not from multithreading.

1.2 Inputs

The program will accept two dense matrices of floating-point values:

- Matrix A with dimensions $M \times K$.
- Matrix B with dimensions $K \times N$.

The program will calculate their product, matrix C, with dimensions $M \times N$, where each output element is defined as:

$C[i,j] = \Sigma\ A[i,k] \times B[k,j]$, for $k = 0$ to $K - 1$.

The input matrices will use 32-bit floating-point values stored in row-major order. The values will be random numbers between –1 and 1, generated from a fixed seed so that every run uses the same inputs.

The matrices will be tested across a range of square sizes ($M = K = N$), from small matrices that fit in the CPU cache to large matrices that do not. Some sizes will not be multiples of 4 or the tile size, to test whether the program handles leftover elements correctly.

1.3 Proposed Processes

1.3.1 Sequential Implementation

The implementation will use the i-k-j loop order. For each row of A, it takes one element A[i][k] and multiplies it with the whole row k of B, adding the result to row i of C. This version will establish the baseline execution time and provide reference results for correctness validation.

1.3.2 SIMD Implementation

The second implementation will use handwritten ARM64 NEON assembly instructions to perform multiple numerical operations in parallel within a vector instruction, combined with cache blocking (tiling) and register reuse.

The target is ARM NEON assembly on Apple Silicon, based on the hardware available to our group and upon feedback.

To maximize performance and avoid memory bottlenecks, the SIMD implementation combines three core techniques:

1. **Cache Blocking (Tiling) -** The outer loops in C/C++ split matrices A, B, and C into smaller sub-matrix blocks (tiles) that fit in the CPU cache so that loaded data can be reused before being evicted.
2. **Register Reuse -** Instead of reading C from memory and writing it back at every step of k, the inner assembly kernel keeps a small 4 × 4 block of C in four NEON registers while iterating across k, writing the accumulated results back to memory only once the loop finishes.
3. **Handwritten NEON Assembly Vectorization -** Each NEON register holds 4 floats, so the inner assembly kernel computes 4 elements per row of the C block at once:
    - Load elements of A and 4 consecutive elements of row k of B into NEON registers.
    - Multiply and add all 4 lanes into the registers holding the block of C using the NEON fused multiply-add instruction (*fmla*).
    - Store the 4 results back to C after accumulating across the tile.

If the number of rows or columns is not a multiple of 4 or the tile size, the leftover elements will be computed with regular scalar code.

1.3.3 Processes to Be Parallelized

The parallelized process is the inner multiply-and-add accumulation kernel inside the cache-blocked loops, written in handwritten ARM64 NEON assembly. Each vector instruction computes 4 elements of C at once instead of 1 while reusing registers across the k loop.

1.3.4 Processes Not to Be Parallelized

The following processes will remain outside the main SIMD computation:

- Input matrix generation and initialization.
- Memory allocation and output buffer initialization.
- Input dimension validation.
- Comparison of the sequential and SIMD output matrices.
- Calculation and presentation of benchmark results.
- Multithreading. All versions run on one CPU core.

These processes are not the primary focus of the project and will be kept separate from the matrix multiplication routines to simplify correctness testing and performance measurement.

1.4 Existing Implementations and Differences

Matrix multiplication is a well-established operation in numerical computing. Existing implementations include Apple's Accelerate framework, which provides optimized numerical computing routines on Apple

platforms, and Intel oneAPI Math Kernel Library, which provides optimized mathematical routines for supported Intel architectures.

Goto and van de Geijn (2008) explained how fast matrix multiplication libraries work. They split the matrices into blocks that fit in the cache and use a small, highly optimized inner kernel. BLIS (Van Zee & van de Geijn, 2015) built on this idea so that only a small micro-kernel needs to be rewritten for each processor. More recently, Alaejos et al. (2024) showed that NEON micro-kernels can be generated automatically and perform close to libraries such as BLIS and OpenBLAS.

Our project is different from simply calling a pre-built library because we will write the scalar version, the cache-blocking loops, and the handwritten ARM64 NEON assembly kernel with register reuse ourselves to show how SIMD and memory-hierarchy optimizations speed up matrix multiplication. We will not implement complex library-level buffer packing, as the goal is not to outperform commercial libraries.

We will also run Apple's Accelerate framework as a reference for the best achievable speed. Accelerate can use Apple's AMX matrix coprocessor instead of only NEON (Hübner et al., 2025), so it is not a direct NEON-to-NEON comparison.

1.5 Parallelization Plan

The project will be implemented through the following steps:

1. **Developing the sequential baseline** - Implement matrix multiplication in C/C++ using standard loops.
2. **Validating the baseline** - Test the implementation using small matrices with known results.
3. **Developing the SIMD implementation** - Implement cache blocking (tiling) in C/C++ and write the inner micro-kernel using handwritten ARM64 NEON assembly with register reuse to process multiple numerical values per vector operation.
4. **Handling vector and tile boundaries** - Ensure correct results when matrix dimensions are not divisible by the vector width (4) or the tile size.
5. **Validating both implementations** - Compare the SIMD output with the sequential output. Because floating-point rounding can differ slightly, results will pass if their relative difference is within a small tolerance.
6. **Benchmarking execution time**:
    - Time only the matrix multiplication, not setup or checking.
    - Run each version multiple times per matrix size after a warm-up run, and report the average.
    - Build the sequential version with automatic vectorization turned off, so the compiler does not automatically add SIMD to it.
7. **Analyzing the results** - For each size, report execution time, speedup, and GFLOPS. Compare the measured speedup with the theoretical maximum and explain how vectorization, register reuse, and cache blocking affect performance across different matrix sizes.

The main performance metrics will be:

- **Speedup = Sequential execution time / SIMD execution time.**
- **GFLOPS = $(2 \times N^3)$ / (execution time in seconds $\times 10^9$)**

The evaluation will report execution times for the sequential and SIMD implementations, output correctness, and speedup. Our group will not assume that SIMD will always be faster, since the results will be interpreted based on the actual measurements.

1.6 Outputs

The project will produce:

- A sequential matrix multiplication implementation.
- A SIMD-vectorized matrix multiplication implementation using handwritten ARM64 NEON assembly, cache blocking, and register reuse.
- Correctness-validation results comparing both outputs.
- Execution-time measurements across multiple matrix sizes.
- Speedup and GFLOPS calculations.
- A discussion of the parallel algorithms, cache blocking, register reuse, and the limitations of SIMD vectorization.
- A GitHub repository containing the source code, build instructions, benchmark results, and documentation.

The expected outcome is a working proof of concept that demonstrates data-level parallelism in a numerical operation relevant to neural-network workloads.

2. Proposed Target Implementation Platform

Our group members use Apple Silicon MacBooks and do not have guaranteed access to NVIDIA GPUs, HPC resources, paid cloud environments, or native x86-64 systems supporting AVX2.

The proposed implementation platform is:

- **Programming language:** C/C++ and handwritten ARM64 NEON assembly.
- **Operating system:** macOS.
- **Processor architecture:** ARM64 / Apple Silicon.
- **Proposed SIMD instruction set:** ARM NEON, using handwritten ARM64 assembly instructions for the inner kernel.
- **Development tools:** Clang/LLVM and the macOS command-line development tools.
- **Reference library:** Apple Accelerate, used only as a performance reference.
- **Version control:** Git and GitHub.
- **Performance evaluation:** Repeated local execution-time measurements using consistent inputs and compiler settings.

Following feedback, our group will use ARM NEON on Apple Silicon with handwritten ARM64 SIMD assembly rather than C intrinsics. The project will not require neural-network training, large datasets, or

paid computing infrastructure. Input data will be generated locally, and both implementations will be evaluated on the same machine wherever possible.

3. Mid-Project Milestones

Milestone 1 - Week 6: Project Proposal

Our group will submit the revised project proposal, define the sequential and SIMD algorithms (including cache blocking and register reuse), identify the target implementation platform, and establish the correctness and benchmarking procedures.

Expected deliverables:

- Revised project proposal document.
- Short video pitch (5-8 minutes) of the proposal.
- Defined algorithms and evaluation methodology.
- Initial public GitHub repository.
- Proposed implementation platform and handwritten ARM64 NEON assembly approach.

Milestone 2 - Week 10: Mid-Project Update and Demonstration

By Week 10, our group will have completed the sequential implementation and the handwritten ARM64 NEON assembly SIMD implementation with register reuse and cache blocking.

Expected deliverables:

- Working sequential implementation.
- Working handwritten ARM64 NEON assembly SIMD implementation with cache blocking, register reuse, and scalar handling of leftover elements.
- Correctness results showing that the SIMD output matches the sequential output for all tested sizes, including sizes that are not multiples of 4.
- Preliminary execution times and speedup for the tested matrix sizes.
- Mid-project demonstration and identification of remaining work.

Milestone 3 - Week 13: Final Submission and Demonstration

By Week 13, our group will complete correctness testing, full performance evaluation across matrix sizes, documentation, and the final demonstration.

Expected deliverables:

- Complete source code in GitHub.
- Build and execution instructions.
- Discussion of the sequential and SIMD algorithms, including cache blocking and register reuse.

- Correctness-validation results.
- Execution-time comparison between the sequential and SIMD implementations, with speedup and GFLOPS.
- Comparison with the Apple Accelerate reference.
- Discussion of performance limitations and possible improvements.
- Final project demonstration.

4. References

- Alaejos, G., Martínez, H., Castelló, A., Dolz, M. F., Igual, F. D., Alonso-Jordá, P., & Quintana-Ortí, E. S. (2024). Automatic generation of ARM NEON micro-kernels for matrix multiplication. *The Journal of Supercomputing*, 80(10), 13873–13899. https://doi.org/10.1007/s11227-024-05955-8
- Apple Inc. (n.d.). *Accelerate*. Apple Developer Documentation. Retrieved https://developer.apple.com/documentation/accelerate
- Arm Limited. (2024). *Arm Architecture Reference Manual for A-profile architecture* (DDI 0487). Arm Developer. Retrieved https://developer.arm.com/documentation/ddi0487/latest
- Arm Limited. (n.d.). *Neon Intrinsics Reference*. Arm Developer. Retrieved https://developer.arm.com/architectures/instruction-sets/intrinsics/
- Ferruz, N., Schmidt, S., & Höcker, B. (2022). ProtGPT2 is a deep unsupervised language model for protein design. *Nature Communications*, 13(1), 4348. https://doi.org/10.1038/s41467-022-32007-7
- Goto, K., & van de Geijn, R. A. (2008). Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3), 1–25. https://doi.org/10.1145/1356052.1356053
- Hübner, P., Hu, A., Peng, I., & Markidis, S. (2025). *Apple vs. Oranges: Evaluating the Apple Silicon M-Series SoCs for HPC Performance and Efficiency*. https://arxiv.org/abs/2502.05317
- Intel Corporation. (n.d.-a). *Intel 64 and IA-32 Architectures Software Developer's Manual*. Intel. Retrieved https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- Intel Corporation. (n.d.-b). *Intel Intrinsics Guide*. Intel. Retrieved https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html
- Intel Corporation. (n.d.-c). *Intel oneAPI Math Kernel Library (oneMKL)*. Intel. Retrieved https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html
- Van Zee, F. G., & van de Geijn, R. A. (2015). BLIS: A Framework for Rapidly Instantiating BLAS Functionality. *ACM Transactions on Mathematical Software*, 41(3), 1–33. https://doi.org/10.1145/2764454